

Document AI Chatbot

RAG-Based Study Assistant

Complete Project Report
& Interview Preparation Guide

Author: Mani

Technology: Python · FastAPI · LangChain · ChromaDB · Gemini

Repository: github.com/mani30mk/Document-AI-Chatbot

July 2026

Contents

1	Project Overview	2
1.1	What Is This Project?	2
1.2	Key Features	2
1.3	Motivation & Problem Statement	2
2	System Architecture	3
2.1	High-Level Architecture Diagram	3
2.2	Component Breakdown	3
3	The RAG Pipeline — Deep Dive	3
3.1	Step 1 — Document Ingestion	4
3.2	Step 2 — Text Chunking	4
3.3	Step 3 — Embedding Generation	5
3.4	Step 4 — Vector Storage (ChromaDB)	5
3.5	Step 5 — Retrieval	6
3.6	Step 6 — Prompt Construction & Generation	6
3.7	Step 7 — Conversation Memory Management	7
4	Backend — FastAPI Deep Dive	7
4.1	API Endpoints Summary	7
4.2	Session Management	8
4.3	CORS Configuration	8
4.4	Pydantic Request/Response Models	8
4.5	Error Handling Strategy	9
5	Frontend — UI Deep Dive	9
5.1	Layout	9
5.2	Design System	9
5.3	Key JavaScript Functions	9
6	Technology Stack — Detailed	10
6.1	FastAPI	10
6.2	LangChain	10
6.3	ChromaDB	10
6.4	Google Gemini 2.5 Flash	10
6.5	HuggingFace Sentence Transformers	11
6.6	Dependencies (requirements.txt)	11
7	Testing	11
8	Project Structure	12
9	How to Run	12

10 Interview Questions & Answers	13
10.1 General Project Questions	13
10.2 RAG & LangChain Questions	14
10.3 FastAPI & Backend Questions	18
10.4 Embeddings & Vector Database Questions	20
10.5 LLM & Gemini Questions	22
10.6 Frontend Questions	23
10.7 Advanced / Design Decision Questions	24
10.8 Python & Software Engineering Questions	27
10.9 Testing & Quality Questions	28
10.10 Scenario-Based Questions	30
10.11 Conceptual / Theory Questions	33
11 Future Improvements	35
12 Conclusion	35

1 Project Overview

1.1 What Is This Project?

Document AI Chatbot is a full-stack **Retrieval-Augmented Generation (RAG)** application that allows users to upload study materials (PDF, DOCX, PPTX, TXT), view them natively in the browser, and interact with them through a smart chatbot powered by **Google Gemini**.

The system ingests documents, splits them into semantically meaningful chunks, embeds them into a vector space using **HuggingFace Sentence Transformers**, stores them in a **ChromaDB** vector database, and retrieves the most relevant chunks at query time to generate grounded, context-aware answers via **Gemini 2.5 Flash**.

1.2 Key Features

1. **RAG Chatbot:** Retrieves top- k relevant document chunks and passes them as context to the LLM for grounded Q&A.
2. **Native File Viewer:** PDFs and TXT files render directly in the browser alongside the chatbot.
3. **One-Click Summarization:** Generates comprehensive summaries of entire documents using Gemini's massive context window.
4. **Multi-Format Support:** Supports .pdf, .txt, .docx, and .pptx files.
5. **Multi-Turn Conversation Memory:** Maintains chat history across turns (up to 10 turns / 20 messages).
6. **Per-Session Isolation:** Each session gets its own vector store, chat history, and file registry.

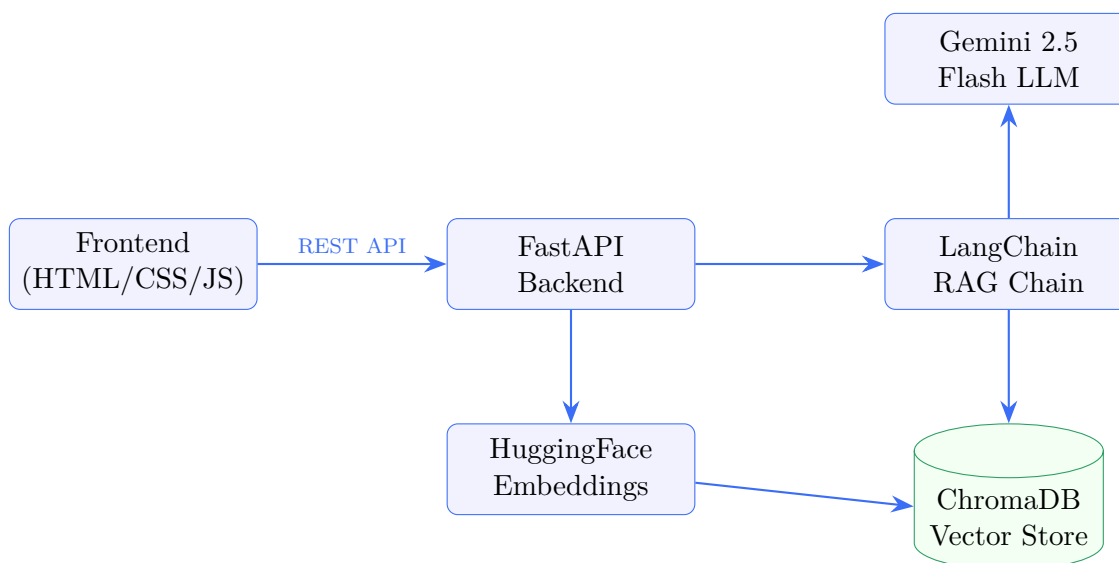
1.3 Motivation & Problem Statement

Students and professionals often need to quickly extract information from large documents. Traditional search is keyword-based and misses semantic meaning. This project solves that by combining:

- **Semantic search** (vector embeddings) for intelligent retrieval.
- **Generative AI** (Gemini) for natural-language, context-aware answers.
- **Conversation memory** for follow-up questions.

2 System Architecture

2.1 High-Level Architecture Diagram



2.2 Component Breakdown

Component	Technology	Purpose
Frontend	Vanilla HTML/CSS/JS	User interface: file upload, preview, chat
Backend API	FastAPI (Python)	REST endpoints, session management, file processing
Document Loaders	LangChain Community	Parse PDF, DOCX, PPTX, TXT into text
Text Splitter	RecursiveCharacterTextSplitter	Split documents (500 chars, 50 overlap)
Embeddings	HuggingFace all-MiniLM-L6-v2	Convert text to 384-dim vectors
Vector Database	ChromaDB	Store and retrieve embeddings via similarity search
LLM	Google Gemini 2.5 Flash	Generate answers from retrieved context
Prompt Template	LangChain ChatPromptTemplate	Structure system/user/history messages
Conversation Memory	In-memory list Python	Store HumanMessage/AIMessage objects

3 The RAG Pipeline — Deep Dive

RAG (Retrieval-Augmented Generation) is the core architectural pattern of this project. It augments the LLM's generation with relevant retrieved context, reducing hallucination

and grounding responses in the user's own documents.

3.1 Step 1 — Document Ingestion

When a user uploads files via the `/upload/{session_id}` endpoint:

1. **Temp File Creation:** Each uploaded file is written to a temporary file on disk (using Python's `tempfile.NamedTemporaryFile`).
2. **Format-Specific Loading:** The correct LangChain loader is selected based on file extension:
 - `.pdf` → `PyPDFLoader`
 - `.docx` → `UnstructuredWordDocumentLoader`
 - `.pptx` → `UnstructuredPowerPointLoader`
 - `.txt` → `TextLoader`
3. **Text Extraction:** Each loader returns a list of `Document` objects containing `page_content` and `metadata`.
4. **Cleanup:** The temporary file is deleted via `os.unlink()`.

Listing 1: Document loading logic in `main.py`

```
1 def load_file(path: str, ext: str):
2     loaders = {
3         ".pdf": PyPDFLoader,
4         ".docx": UnstructuredWordDocumentLoader,
5         ".pptx": UnstructuredPowerPointLoader,
6         ".txt": TextLoader,
7     }
8     loader_cls = loaders.get(ext)
9     if not loader_cls:
10         raise ValueError(f"Unsupported file type: {ext}")
11     return loader_cls(path).load()
```

3.2 Step 2 — Text Chunking

Documents are split into overlapping chunks using `RecursiveCharacterTextSplitter`:

- **Chunk Size:** 500 characters
- **Chunk Overlap:** 50 characters
- **Why Recursive?** It tries to split by paragraphs → sentences → words → characters, preserving semantic coherence.
- **Why Overlap?** Ensures context at chunk boundaries is not lost.

Listing 2: Text splitter configuration

```
1 TEXT_SPLITTER = RecursiveCharacterTextSplitter(  
2     chunk_size=500,  
3     chunk_overlap=50  
4 )  
5 chunks = TEXT_SPLITTER.split_documents(all_docs)
```

3.3 Step 3 — Embedding Generation

Each chunk is embedded using the `all-MiniLM-L6-v2` model from Sentence Transformers:

- **Model:** `sentence-transformers/all-MiniLM-L6-v2`
- **Embedding Dimension:** 384
- **Why This Model?** Lightweight (22M parameters), fast inference, good semantic quality for retrieval tasks. Runs locally without GPU.
- **Singleton Pattern:** The embedding model is loaded once and cached globally to avoid repeated initialization overhead.

Listing 3: Embedding initialization with caching

```
1 EMBEDDINGS = None  
2 EMBEDDINGS_ERROR = None  
3  
4 def get_embeddings():  
5     global EMBEDDINGS, EMBEDDINGS_ERROR  
6     if EMBEDDINGS is not None:  
7         return EMBEDDINGS  
8     if EMBEDDINGS_ERROR is not None:  
9         raise EMBEDDINGS_ERROR  
10    try:  
11        EMBEDDINGS = HuggingFaceEmbeddings(  
12            model_name="sentence-transformers/all-MiniLM-L6-v2"  
13        )  
14        return EMBEDDINGS  
15    except Exception as exc:  
16        EMBEDDINGS_ERROR = exc  
17        raise
```

3.4 Step 4 — Vector Storage (ChromaDB)

Embeddings are stored in ChromaDB, a lightweight, in-memory vector database:

- **Per-Session Stores:** Each session gets its own ChromaDB collection, ensuring data isolation.
- **Additive Uploads:** If a session already has a vector store, new chunks are appended via `add_texts()`.

- **First Upload:** Creates a new collection via `Chroma.from_documents()`.

Listing 4: Vector store creation/update

```
1 if session_id in session_vectorstores:
2     session_vectorstores[session_id].add_texts(
3         [c.page_content for c in chunks]
4     )
5 else:
6     session_vectorstores[session_id] = Chroma.from_documents(
7         chunks, embeddings
8     )
```

3.5 Step 5 — Retrieval

When the user asks a question via the `/ask` endpoint:

1. The vector store is converted to a **retriever** with `k=4`.
2. The user's question is embedded using the same `all-MiniLM-L6-v2` model.
3. ChromaDB performs a **cosine similarity search** and returns the top 4 most relevant chunks.

Listing 5: Retrieval

```
1 retriever = session_vectorstores[sid].as_retriever(
2     search_kwargs={"k": 4}
3 )
4 retrieved_docs = retriever.invoke(req.question)
5 context = format_docs(retrieved_docs)
```

3.6 Step 6 — Prompt Construction & Generation

The retrieved context, chat history, and user question are assembled into a structured prompt:

Listing 6: RAG prompt template

```
1 RAG_PROMPT = ChatPromptTemplate.from_messages([
2     (
3         "system",
4         """You are a helpful study assistant. Answer questions
5 using ONLY the context below.
6 If the answer isn't in the context, say so honestly.
7
8 Context:
9 {context}""",
10    ),
11    MessagesPlaceholder(variable_name="chat_history"),
12    ("human", "{question}"),
13 ])
```

The chain is: RAG_PROMPT → Gemini LLM → StrOutputParser

Listing 7: Chain execution

```
1 chain = RAG_PROMPT | llm | StrOutputParser()
2 answer = chain.invoke({
3     "context": context,
4     "chat_history": history,
5     "question": req.question,
6 })
```

3.7 Step 7 — Conversation Memory Management

- Each user message is stored as `HumanMessage` and each AI response as `AIMessage`.
- History is **truncated to 20 messages** (10 turns) to prevent token overflow.
- The `MessagesPlaceholder` in the prompt template injects the full history into each subsequent call.

Listing 8: History management

```
1 session_histories[sid].append(HumanMessage(content=req.question))
2 session_histories[sid].append(AIMessage(content=answer))
3
4 # Keep last 20 messages (10 turns)
5 if len(session_histories[sid]) > 20:
6     session_histories[sid] = session_histories[sid][-20:]
```

4 Backend — FastAPI Deep Dive

4.1 API Endpoints Summary

Method	Endpoint	Description
POST	/session/new	Create a new session; returns <code>session_id</code>
POST	/upload/{session_id}	Upload files (multipart); returns chunk count
POST	/ask	Ask a question (JSON body); returns RAG answer
POST	/summarize	Summarize a specific uploaded file
DELETE	/session/{session_id}/history	Delete chat history for a session
GET	/session/{session_id}	Get session info (files, history, vectorstore status)
GET	/health	Health check endpoint

4.2 Session Management

The application uses four in-memory dictionaries to manage session state:

Listing 9: Session state stores

```
1 session_vectorstores: dict[str, Chroma] = {}      # Vector DBs
2 session_histories:    dict[str, list]    = {}      # Chat history
3 session_files:        dict[str, list[str]] = {}    # Uploaded
                        filenames
4 session_docs:          dict[str, dict[str, str]] = {} # Full text
                        for summarization
```

Key Point

In production, these should be replaced with Redis or a proper session store for persistence and horizontal scaling.

4.3 CORS Configuration

Listing 10: CORS middleware

```
1 app.add_middleware(
2     CORSMiddleware,
3     allow_origins=["*"],      # tighten in production
4     allow_methods=["*"],
5     allow_headers=["*"],
6 )
```

This allows the frontend (served as a static HTML file via `file://` or any origin) to communicate with the FastAPI backend.

4.4 Pydantic Request/Response Models

Listing 11: API models

```
1 class AskRequest(BaseModel):
2     session_id: str
3     question: str
4     gemini_api_key: str
5
6 class AskResponse(BaseModel):
7     answer: str
8     session_id: str
9     sources_used: int
10
11 class SummarizeRequest(BaseModel):
12     session_id: str
13     filename: str
14     gemini_api_key: str
```

4.5 Error Handling Strategy

- **404:** Session not found (invalid or expired `session_id`).
- **400:** No documents loaded, or no files could be parsed.
- **503:** Embedding model failed to initialize (e.g., missing `sentence-transformers`).
- **500:** LLM/Summarization errors caught and returned with details.
- Unsupported file types return an inline error response (not an HTTP error).

5 Frontend — UI Deep Dive

The frontend is a single `index.html` file (534 lines) containing all HTML, CSS, and JavaScript.

5.1 Layout

- **Top Navigation Bar:** Logo, API key input, Connect button, connection status indicator.
- **Sidebar (260px):** File upload zone (drag-and-drop + click), uploaded file list with type badges.
- **Main Viewer:** Displays file previews (PDF via `<iframe>`, TXT via `FileReader`).
- **Chat Panel:** Floating chat window (FAB button), message bubbles, typing indicators.

5.2 Design System

- **CSS Custom Properties:** 14 design tokens (colors, radii, shadows) defined in `:root`.
- **Dark Mode:** Automatic via `@media (prefers-color-scheme: dark)`.
- **Responsive:** CSS Grid layout (`grid-template-columns: 260px 1fr`).

5.3 Key JavaScript Functions

Function	Purpose
<code>initSession()</code>	Reads API key, calls <code>/session/new</code> , updates connection status
<code>uploadFiles()</code>	Sends files via <code>FormData</code> to <code>/upload/{sid}</code> , updates file list
<code>selectFile(i)</code>	Renders file preview (PDF <code>iframe</code> / TXT reader / info card)
<code>sendMessage()</code>	Posts question to <code>/ask</code> , displays answer bubble

Function	Purpose
<code>summarizeFile()</code>	Calls <code>/summarize</code> , displays summary overlay
<code>clearHistory()</code>	DELETE request to clear conversation memory
<code>toggleChat()</code>	Shows/hides the floating chat window

6 Technology Stack — Detailed

6.1 FastAPI

- Modern, async-capable Python web framework.
- Automatic OpenAPI/Swagger documentation at `/docs`.
- Built-in request validation via Pydantic.
- ASGI server (Uvicorn) for high concurrency.

6.2 LangChain

- Framework for building LLM-powered applications.
- Provides abstractions for: document loading, text splitting, embeddings, vector stores, prompts, chains.
- Uses LCEL (LangChain Expression Language): `prompt | llm | parser`.
- Sub-packages used: `langchain-community`, `langchain-huggingface`, `langchain-chroma`, `langchain-google-genai`.

6.3 ChromaDB

- Open-source, lightweight embedding database.
- Default: in-memory storage (ephemeral).
- Supports cosine, L2, and inner-product distance metrics.
- In this project: in-memory, per-session collections.

6.4 Google Gemini 2.5 Flash

- Google's latest multimodal LLM.
- Large context window (up to 1M tokens).
- Fast inference, cost-effective for production.
- Accessed via `langchain-google-genai` with user-provided API key.
- Temperature set to 0.2 for factual, less creative responses.

6.5 HuggingFace Sentence Transformers

- all-MiniLM-L6-v2: 22M parameters, 384-dim embeddings.
- Trained on 1B+ sentence pairs for semantic similarity.
- Runs locally (no API key needed), CPU-friendly.
- Produces normalized embeddings suitable for cosine similarity.

6.6 Dependencies (requirements.txt)

Package	Purpose
fastapi	Web framework
uvicorn[standard]	ASGI server
python-multipart	File upload support in FastAPI
langchain	Core LangChain framework
langchain-community	Document loaders (PDF, DOCX, PPTX, TXT)
langchain-huggingface	HuggingFace embeddings integration
langchain-chroma	ChromaDB vector store integration
langchain-google-genai	Google Gemini LLM integration
sentence-transformers	Underlying embedding model
pypdf	PDF text extraction
unstructured	DOCX/PPTX parsing engine
python-docx	Word document support
python-pptx	PowerPoint support
chromadb	Vector database

7 Testing

The project includes a test file `test_main.py` using `pytest` and FastAPI's `TestClient`.

Listing 12: Test: embedding failure returns 503

```

1 def test_embeddings_failure_returns_503_on_upload(client):
2     with patch("main.get_embeddings",
3               side_effect=RuntimeError("offline")):
4         response = client.post("/session/new", json={})
5         sid = response.json()["session_id"]
6
7         upload_response = client.post(
8             f"/upload/{sid}",
9             files=[("files",
10                  ("sample.txt", b"hello world", "text/plain"))
11                  ],
12         )
13
14     assert upload_response.status_code == 503
15     assert "Embedding model" in upload_response.json()["detail"]

```

What this tests:

- Verifies that when the embedding model fails to load, the upload endpoint returns HTTP 503 (Service Unavailable).
- Uses `unittest.mock.patch` to simulate the failure.
- Validates both the status code and the error message.

8 Project Structure

Listing 13: Directory structure

```
1 Document-AI-Chatbot/  
2   main.py                # FastAPI backend + RAG logic (316 lines)  
3   index.html             # Frontend SPA (534 lines)  
4   requirements.txt       # Python dependencies (14 packages)  
5   test_main.py           # Pytest unit tests  
6   README.md              # Project documentation  
7   .gitignore             # Git ignore rules  
8   .venv/                 # Virtual environment (not committed)
```

9 How to Run

1. Clone:

```
1 git clone https://github.com/mani30mk/Document-AI-Chatbot.git  
2 cd Document-AI-Chatbot
```

2. Virtual environment:

```
1 python -m venv .venv  
2 .venv\Scripts\Activate.ps1      # Windows PowerShell
```

3. Install dependencies:

```
1 pip install -r requirements.txt
```

4. Start backend:

```
1 uvicorn main:app --reload --port 8000
```

5. Open frontend: Double-click `index.html` in your browser. Enter your Gemini API key and click “Connect”.

10 Interview Questions & Answers

This section contains 50+ interview questions organized by topic. Each question includes a detailed, interview-ready answer.

10.1 General Project Questions

Q1: Can you give a brief overview of your project?

Document AI Chatbot is a full-stack Retrieval-Augmented Generation (RAG) application. Users upload study documents (PDF, DOCX, PPTX, TXT), which are parsed, chunked into 500-character segments with 50-character overlap, embedded using HuggingFace’s all-MiniLM-L6-v2 model (384 dimensions), and stored in per-session ChromaDB vector stores. When a user asks a question, the system retrieves the top-4 most semantically similar chunks via cosine similarity, injects them as context into a structured prompt, and sends it to Google Gemini 2.5 Flash for answer generation. The backend is built with FastAPI and LangChain, and the frontend is a single-page HTML/CSS/JS application with a floating chat widget.

Q2: Why did you choose RAG over fine-tuning?

Fine-tuning requires retraining the model on domain-specific data, which is expensive, time-consuming, and inflexible—every time new documents are added, you’d need to retrain. RAG, on the other hand, decouples knowledge from the model: the LLM stays general-purpose while document-specific knowledge is stored in a vector database that can be updated instantly. RAG also provides **traceability**—I can show which chunks were used to generate an answer—and **reduces hallucination** by grounding responses in actual document content. For a study assistant where documents change frequently, RAG is the clear choice.

Q3: What real-world problem does this solve?

Students and professionals often need to extract specific information from large documents quickly. Traditional keyword search misses semantic meaning (e.g., searching “salary” won’t find “compensation” or “remuneration”). This project uses semantic vector search to understand meaning, not just keywords, and generates natural-language answers. It’s like having a personal tutor who has read all your materials and can answer questions with context.

Q4: What are the limitations of your current implementation?

1. **In-memory state:** Sessions are lost on server restart (no persistence).
2. **No authentication:** Any user can create sessions; API keys are sent in plain JSON.
3. **Single-server:** In-memory dicts can't scale horizontally.
4. **No streaming:** Answers come as a single response (no token-by-token streaming).
5. **CORS wildcard:** `allow_origins=["*"]` is too permissive for production.
6. **Fixed chunk parameters:** 500-char chunks may not be optimal for all document types.
7. **No re-ranking:** Retrieved chunks aren't re-ranked before being sent to the LLM.

Q5: How would you deploy this to production?

1. **Backend:** Containerize with Docker, deploy on AWS ECS / GCP Cloud Run / Kubernetes.
2. **Session State:** Replace in-memory dicts with Redis for persistence and horizontal scaling.
3. **Vector DB:** Switch from in-memory ChromaDB to a managed service like Pinecone, Weaviate, or persistent ChromaDB with a disk-backed SQLite.
4. **Frontend:** Serve via CDN (CloudFront, Vercel).
5. **Security:** Add OAuth2/JWT authentication; store API keys server-side encrypted; restrict CORS origins.
6. **Monitoring:** Add LangSmith for LLM observability, Prometheus/Grafana for metrics.
7. **Streaming:** Use Server-Sent Events (SSE) or WebSockets for token-by-token responses.

10.2 RAG & LangChain Questions

Q6: Explain the RAG pipeline step by step.

1. **Ingestion:** Documents are uploaded and parsed using format-specific LangChain loaders (PyPDFLoader, UnstructuredWordDocumentLoader, etc.).
2. **Chunking:** Text is split into 500-character chunks with 50-character overlap using RecursiveCharacterTextSplitter.
3. **Embedding:** Each chunk is converted to a 384-dimensional vector using the all-MiniLM-L6-v2 sentence transformer model.
4. **Storage:** Vectors are stored in a per-session ChromaDB collection.
5. **Retrieval:** When a user asks a question, the question is embedded using the same model, and the top-4 most similar chunks are retrieved via cosine similarity.
6. **Augmentation:** Retrieved chunks are formatted and injected into the system prompt as “Context”.
7. **Generation:** The full prompt (system + context + chat history + question) is sent to Gemini 2.5 Flash, which generates a grounded answer.

Q7: Why did you use RecursiveCharacterTextSplitter instead of other splitters?

`RecursiveCharacterTextSplitter` is the most versatile splitter because it uses a hierarchy of separators: `["\n\n", "\n", " ", ""]`. It first tries to split by double newlines (paragraphs), then single newlines, then spaces, then individual characters. This preserves semantic boundaries as much as possible within the chunk size constraint. Alternatives like `CharacterTextSplitter` only split on a single separator, which can break mid-sentence. `TokenTextSplitter` is token-based (useful for precise token budgets) but slower. For a general-purpose study assistant handling diverse document types, `RecursiveCharacterTextSplitter` offers the best balance.

Q8: Why chunk size 500 and overlap 50? How would you tune these?

500 characters is approximately 100–125 tokens—small enough to be semantically focused but large enough to contain a complete thought. **50-character overlap** ensures that information at chunk boundaries isn't lost. To tune:

- **Increase chunk size** (1000+) for documents with long, cohesive paragraphs (e.g., legal contracts).
- **Decrease chunk size** (200–300) for FAQ-style or highly structured content.
- **Evaluate empirically:** Use retrieval metrics like precision@k, recall@k, or MRR (Mean Reciprocal Rank) on a test set.
- Consider **semantic chunking** (splitting by topic/meaning) for more advanced use cases.

Q9: What is LCEL and how does your project use it?

LCEL (LangChain Expression Language) is a declarative way to compose LangChain components using the pipe (`|`) operator. In my project, the chain is:

```
RAG_PROMPT | llm | StrOutputParser()
```

This means: take the prompt template, pass its output to the LLM, then parse the LLM's output to a plain string. LCEL supports streaming, batching, and async out of the box. It replaces the older `LLMChain` and `SequentialChain` patterns.

Q10: How does the conversation memory work?

I use a manual list-based approach with LangChain's `HumanMessage` and `AIMessage` objects. After each Q&A turn:

1. The question is appended as a `HumanMessage`.
2. The answer is appended as an `AIMessage`.
3. If the history exceeds 20 messages (10 turns), the oldest messages are trimmed.

The history is injected into the prompt via `MessagesPlaceholder(variable_name="chat_history")`. This lets the LLM understand follow-up questions like “What about the second point?” by seeing the prior conversation context.

Q11: Why top-k=4 for retrieval? How would you improve retrieval quality?

k=4 is a practical default—enough context for most questions without overwhelming the LLM or exceeding token limits. To improve retrieval:

1. **Re-ranking:** Use a cross-encoder model (e.g., `ms-marco-MiniLM`) to re-rank the top-20 retrieved chunks and select the best 4.
2. **Hybrid search:** Combine dense (vector) retrieval with sparse (BM25/TF-IDF) retrieval for better keyword coverage.
3. **MMR (Maximal Marginal Relevance):** Diversify results to avoid redundant chunks: `search_type="mmr"`.
4. **Metadata filtering:** Filter chunks by source file or page number.
5. **Query expansion:** Use the LLM to generate multiple query reformulations before retrieval.

Q12: Explain what cosine similarity is and why it's used for retrieval.

Cosine similarity measures the angle between two vectors in a high-dimensional space:

$$\text{cosine_sim}(A, B) = \frac{A \cdot B}{\|A\| \times \|B\|}$$

It ranges from -1 (opposite) to 1 (identical direction). It's used for retrieval because:

- It's **magnitude-invariant**: only considers the direction (meaning), not the length of the vector.
- Semantically similar texts produce vectors pointing in similar directions, regardless of document length.
- The all-MiniLM-L6-v2 model produces normalized embeddings, so cosine similarity simplifies to a dot product—very fast to compute.

Q13: What is the difference between an embedding and a vector?

A **vector** is a general mathematical object—an ordered list of numbers. An **embedding** is a specific type of vector that is the *learned representation* of data (text, images, etc.) in a continuous vector space, where semantic similarity maps to geometric proximity. In this project, each text chunk is converted to a 384-dimensional embedding where chunks about similar topics end up close together in vector space.

Q14: How does the system prompt prevent hallucination?

The system prompt explicitly instructs: “*Answer questions using ONLY the context below. If the answer isn’t in the context, say so honestly.*” This is a **grounding constraint**—it tells the LLM to use only the retrieved context, not its pretrained knowledge. When the answer isn’t in the retrieved chunks, the model is instructed to admit it rather than fabricate an answer. This is a key RAG pattern called “faithfulness to context.”

Q15: What happens if no relevant chunks are found?

ChromaDB always returns exactly k results regardless of relevance scores—there’s no built-in threshold. So even for an unrelated question, 4 chunks are returned. However:

1. The system prompt says “if the answer isn’t in the context, say so honestly,” so the LLM should respond appropriately.
2. To improve this, I could add a **relevance score threshold**: filter out chunks with cosine similarity below, say, 0.3, and if no chunks pass the threshold, return a message like “I couldn’t find relevant information in your documents.”

10.3 FastAPI & Backend Questions

Q16: Why FastAPI over Flask or Django?

1. **Async-native:** FastAPI is built on ASGI (Starlette), supporting `async/await` natively—critical for I/O-bound operations like LLM API calls.
2. **Automatic validation:** Pydantic models validate request bodies, query parameters, and path parameters with zero boilerplate.
3. **Auto-generated docs:** Swagger UI at `/docs` and ReDoc at `/redoc` are generated automatically from type hints.
4. **Performance:** One of the fastest Python frameworks, comparable to Go/Node.js for I/O-bound tasks.
5. Flask lacks native async support and requires extensions for validation. Django is heavy for a microservice.

Q17: Explain how file upload works in your backend.

The `/upload/{session_id}` endpoint accepts `multipart/form-data` using FastAPI's `UploadFile` type:

1. Files arrive as `List[UploadFile]` via the `files` form field.
2. Each file is written to a **temporary file** using `tempfile.NamedTemporaryFile` because LangChain loaders require a file path (they can't read from memory).
3. The appropriate loader is selected based on file extension.
4. After loading, the temp file is **deleted** via `os.unlink()` in a `finally` block to ensure cleanup even if parsing fails.
5. The full text is stored in `session_docs` for the summarization feature.

Q18: Why use `tempfile` instead of reading files directly into memory?

LangChain document loaders (`PyPDFLoader`, `UnstructuredWordDocumentLoader`, etc.) are designed to accept **file paths**, not file objects or byte streams. They internally open the file from disk. So we must write the uploaded bytes to a temporary file, let the loader read it, and then clean up. An alternative would be to write custom loaders that accept byte streams, but using `tempfile` is simpler and leverages the existing, well-tested loader ecosystem.

Q19: What is CORS and why is it needed here?

CORS (Cross-Origin Resource Sharing) is a browser security mechanism that blocks web pages from making requests to a different origin (domain/port). Since the frontend is served from `file://` or a different port (not 8000), the browser would block API calls to `http://localhost:8000` without CORS headers. The middleware adds headers like `Access-Control-Allow-Origin: *` to tell the browser it's safe to proceed. In production, I'd restrict `allow_origins` to the specific frontend domain.

Q20: How does session management work? What are its weaknesses?

Sessions are managed via four Python dictionaries keyed by UUID. When `/session/new` is called, a UUID is generated and used as the session ID. All subsequent operations (upload, ask, summarize) require this session ID. **Weaknesses:**

- **No persistence:** Server restart loses all sessions.
- **No expiration:** Sessions never expire, causing memory leaks.
- **No horizontal scaling:** In-memory state can't be shared across multiple server instances.
- **No authentication:** Anyone who guesses/intercepts a session ID can access it.

Fix: Use Redis with TTL-based expiration, JWT tokens for auth, and persistent vector DBs like Pinecone.

Q21: What is Pydantic and how do you use it?

Pydantic is a data validation library that uses Python type hints to define schemas. In my project, I use Pydantic's `BaseModel` for:

- **AskRequest:** Validates that `session_id`, `question`, and `gemini_api_key` are all strings.
- **AskResponse:** Ensures the response contains `answer` (str), `session_id` (str), and `sources_used` (int).
- **SummarizeRequest:** Validates the summarization request.

FastAPI automatically returns a 422 error with details if the request body doesn't match the model. This eliminates manual validation code.

Q22: What does `uvicorn main:app --reload` do?

`uvicorn` is an ASGI server. `main:app` tells it to import the `app` object from `main.py`. `--reload` enables auto-reload: the server watches for file changes and restarts automatically (development mode only). `--port 8000` sets the listening port. In production, you'd remove `--reload` and use `--workers N` for multi-process concurrency.

10.4 Embeddings & Vector Database Questions

Q23: Why all-MiniLM-L6-v2 over OpenAI embeddings?

1. **Free:** No API costs; runs locally.
2. **Privacy:** Document content never leaves the user's machine.
3. **Fast:** 22M parameters; sub-second inference on CPU.
4. **Good quality:** Top performer on sentence similarity benchmarks for its size class.
5. **No rate limits:** Unlike OpenAI's API, there are no throttling concerns.
6. Trade-off: OpenAI's `text-embedding-3-large` (3072 dims) may capture more nuance, but at the cost of API latency, cost, and privacy.

Q24: What is ChromaDB and why did you choose it?

ChromaDB is an open-source vector database designed for AI applications. I chose it because:

- **Zero infrastructure:** Runs in-memory with no external server needed.
- **LangChain integration:** `langchain-chroma` provides seamless `from_documents()` and `as_retriever()` methods.
- **Development speed:** Perfect for prototyping without managing a separate database service.
- For production, I'd switch to Pinecone (managed, scalable), Weaviate (open-source, production-ready), or Qdrant (high-performance, Rust-based).

Q25: How does similarity search work internally in ChromaDB?

1. The query text is embedded using the same embedding model.
2. ChromaDB computes the distance (default: L2/Euclidean, but cosine is also supported) between the query embedding and all stored embeddings.
3. The top- k closest embeddings are returned, along with their associated text chunks and metadata.
4. For small datasets (thousands of chunks), this is a brute-force comparison. For larger datasets, ChromaDB uses HNSW (Hierarchical Navigable Small World) indexing for approximate nearest neighbor (ANN) search.

Q26: What is the Singleton pattern in your embedding initialization?

The `get_embeddings()` function implements a lazy singleton: the embedding model is loaded only once on first call and cached in the global `EMBEDDINGS` variable. Subsequent calls return the cached instance. If loading fails, the error is also cached in `EMBEDDINGS_ERROR` to avoid repeated failed initialization attempts. This saves memory and startup time since loading a transformer model involves downloading weights and initializing PyTorch tensors.

10.5 LLM & Gemini Questions

Q27: Why Gemini 2.5 Flash over GPT-4 or Claude?

1. **Large context window:** Up to 1M tokens—crucial for the summarization feature that sends full document text.
2. **Cost-effective:** Flash variant is optimized for speed and cost.
3. **Fast inference:** Lower latency than GPT-4.
4. **Free tier:** Google offers a generous free tier for Gemini API.
5. **Quality:** Competitive with GPT-4 on reasoning and factual tasks.
6. The architecture is model-agnostic—swapping to GPT-4 or Claude requires only changing the LLM class and API key.

Q28: Why temperature 0.2? What happens at 0 vs. 1?

Temperature controls the randomness of token sampling:

- **0 (deterministic):** Always picks the most probable token. Good for factual Q&A but can be repetitive.
- **0.2 (low):** Mostly deterministic with slight variation—ideal for our use case where we want factual, consistent answers from document context.
- **1.0 (creative):** More diverse, creative outputs—useful for brainstorming or creative writing but undesirable for factual retrieval.

For a study assistant, 0.2 ensures the model sticks closely to the retrieved context while maintaining natural language flow.

Q29: How does the summarization feature differ from RAG Q&A?

They are fundamentally different approaches:

- **RAG Q&A:** Retrieves only top- k relevant chunks → generates an answer from partial context. Uses the vector store and retriever.
- **Summarization:** Sends the **entire document text** to Gemini in a single prompt. No vector store involved. Leverages Gemini's 1M-token context window to read the full document and produce a cohesive summary.

This is why `session_docs` stores the full text separately from the chunked vector store.

Q30: What is a prompt template and why use one?

A prompt template is a reusable structure that defines how to construct the input to an LLM. In my project, `ChatPromptTemplate.from_messages()` creates a multi-message prompt with:

1. A **system message** containing instructions and the retrieved context.
2. A **MessagesPlaceholder** for chat history (dynamically injected).
3. A **human message** with the current question.

Benefits: consistency across calls, separation of concerns (logic vs. prompt), easy experimentation with different prompts, and type-safe variable injection.

10.6 Frontend Questions

Q31: Why a single HTML file instead of React/Vue?

The frontend is intentionally lightweight:

- **Zero build step:** No npm, Webpack, or bundling—just double-click to open.
- **Instant deployment:** Can be served from any static file server or CDN.
- **Focus on backend:** This is primarily a backend/AI project; the frontend demonstrates the capability without overengineering.
- At 534 lines, the complexity is manageable in a single file.
- For a production version, I'd use React or Next.js with proper state management.

Q32: How does drag-and-drop file upload work?

The upload zone listens for three DOM events:

1. **dragover**: Prevents the browser's default behavior (opening the file) and adds a visual highlight class.
2. **dragleave**: Removes the highlight.
3. **drop**: Prevents default, extracts files from `e.dataTransfer.files`, and calls `uploadFiles()`.

Files are also uploadable via the hidden `<input type="file">` element triggered by clicking the upload zone (it's wrapped in a `<label>`).

Q33: How does the PDF viewer work?

When a PDF file is selected:

1. A **Blob URL** is created from the original **File** object using `URL.createObjectURL(f)`.
2. This URL is set as the `src` of an `<iframe>`.
3. The browser's native PDF viewer renders the file inside the `iframe`.
4. When a file is removed, `URL.revokeObjectURL()` is called to free memory.

For TXT files, `FileReader.readAsText()` loads the content and displays it in a `<div>` with `white-space: pre-wrap`.

Q34: How does the dark mode work?

The CSS uses `@media (prefers-color-scheme: dark)` to override the `:root` CSS custom properties when the user's OS is set to dark mode. No JavaScript is involved—it's entirely CSS-driven. The design tokens (colors) are defined as CSS variables, so changing them in the media query automatically updates the entire UI.

10.7 Advanced / Design Decision Questions

Q35: How would you handle very large documents (1000+ pages)?

1. **Async processing:** Move file processing to a background task queue (Celery/RQ). Return immediately with a “processing” status and poll for completion.
2. **Batch chunking:** Process documents in batches to avoid memory spikes.
3. **Streaming upload:** Use chunked file uploads for very large files.
4. **Persistent vector store:** Use disk-backed ChromaDB or Pinecone to avoid holding all vectors in memory.
5. **Hierarchical summarization:** For summarization, use a map-reduce or refine strategy instead of sending the entire text (which may exceed even Gemini’s context window).

Q36: How would you add authentication?

1. **JWT Authentication:** Use FastAPI’s `OAuth2PasswordBearer` dependency with JWT tokens.
2. **User registration:** Add a `/auth/register` and `/auth/login` endpoint.
3. **API key security:** Store Gemini API keys encrypted in a database (not sent from the frontend).
4. **Session ownership:** Link sessions to authenticated users; reject requests from non-owners.
5. **Rate limiting:** Use `slowapi` middleware to prevent abuse.

Q37: How would you add streaming responses?

1. Change the LLM to streaming mode: `llm.stream()` instead of `llm.invoke()`.
2. Use FastAPI’s `StreamingResponse` with `text/event-stream` content type (Server-Sent Events).
3. On the frontend, use the `EventSource` API or `fetch` with `ReadableStream` to consume tokens as they arrive.
4. Display tokens incrementally in the chat bubble for a “typing” effect.

Q38: How would you implement re-ranking?

After retrieving the top- k chunks (e.g., top-20), pass each (query, chunk) pair through a cross-encoder model like `cross-encoder/ms-marco-MiniLM-L-6-v2`. Cross-encoders jointly encode the query and passage and output a relevance score. Re-rank by this score and take the top-4. This is more accurate than bi-encoder similarity because the cross-encoder can model fine-grained interactions between the query and passage. In LangChain, use `FlashrankRerank` or `CohereRerank`.

Q39: What is the difference between bi-encoder and cross-encoder?

- **Bi-encoder** (used in this project): Encodes query and document *independently* into vectors, then compares via cosine similarity. Fast ($O(n)$ for n documents). Used for retrieval.
- **Cross-encoder**: Takes (query, document) as a *single input pair* and outputs a relevance score. More accurate but slow ($O(n \times q)$ inference). Used for re-ranking a small candidate set.

Best practice: Use a bi-encoder to retrieve 20–50 candidates, then a cross-encoder to re-rank and select the top- k .

Q40: How would you add hybrid search (dense + sparse)?

Hybrid search combines:

- **Dense retrieval**: Vector similarity (what we currently do). Good at semantic/conceptual matching.
- **Sparse retrieval**: BM25/TF-IDF keyword matching. Good at exact term matching (names, codes, acronyms).

Implementation: Use `BM25Retriever` from LangChain alongside the vector retriever, then merge results using `EnsembleRetriever` with weighted scores (e.g., 0.6 dense + 0.4 sparse). This handles queries where exact keywords matter (e.g., “What is the FERPA policy?”) better than pure semantic search.

Q41: What is Maximal Marginal Relevance (MMR)?

MMR is a retrieval strategy that balances **relevance** and **diversity**. Standard top- k retrieval might return 4 chunks that are all semantically similar (redundant). MMR iteratively selects chunks that are:

1. Highly relevant to the query.
2. Dissimilar from already-selected chunks.

The formula:

$$\text{MMR} = \arg \max_{d_i} \left[\lambda \cdot \text{sim}(d_i, q) - (1 - \lambda) \cdot \max_{d_j \in S} \text{sim}(d_i, d_j) \right]$$

In LangChain: `retriever = vectorstore.as_retriever(search_type="mmr")`.

10.8 Python & Software Engineering Questions

Q42: Why use `async def` for some endpoints but `def` for others?

In FastAPI:

- `async def` is used when the function contains `await` calls (e.g., `await uf.read()` for file uploads). It runs on the main event loop.
- `def` (synchronous) is used when there are no async operations. FastAPI automatically runs these in a thread pool to avoid blocking the event loop.

In my code: `upload_files` and `ask` are async (they use `await`), while `new_session`, `clear_history`, `session_info`, and `health` are synchronous (they do simple dict operations).

Q43: What is the purpose of the `finally` block in the upload handler?

The `finally` block ensures that the temporary file is **always deleted**, even if the loader throws an exception (e.g., corrupt file, unsupported content). Without it, failed uploads would leave orphan temp files on disk, eventually filling up storage. This is a resource cleanup pattern.

```
1 try:
2     docs = load_file(tmp_path, ext)
3 except ValueError as e:
4     return {"error": str(e), "skipped": uf.filename}
5 finally:
6     os.unlink(tmp_path)  # Always runs
```

Q44: How do you handle the Gemini API key securely?

Currently: The API key is entered by the user in the browser and sent with each request in the JSON body. This means:

- The key is in memory only; it's never stored on the server.
- Each user provides their own key—no shared key that could be compromised.

Security concern: The key is sent in plaintext over HTTP (not HTTPS in local development). In production:

- Enforce HTTPS.
- Store keys server-side encrypted in a secrets manager (e.g., AWS Secrets Manager).
- Use a server-side API key with per-user rate limiting.

Q45: What is the `response_model` parameter in FastAPI?

`response_model` tells FastAPI to:

1. **Validate** the response data against the Pydantic model before sending.
2. **Serialize** the response, filtering out any extra fields not in the model.
3. **Document** the response schema in the auto-generated OpenAPI spec.

For example, `@app.post("/ask", response_model=AskResponse)` ensures the response always contains exactly `answer`, `session_id`, and `sources_used`.

10.9 Testing & Quality Questions

Q46: Explain the test in `test_main.py`.

The test verifies that when the embedding model fails to initialize (simulated via `unittest.mock.patch` making `get_embeddings` raise `RuntimeError("offline")`), the upload endpoint returns:

- HTTP status code **503** (Service Unavailable).
- An error message containing “Embedding model”.

This is a **fault injection test**—it validates that the system handles dependency failures gracefully instead of crashing with a 500 error.

Q47: What additional tests would you write?

1. **Session lifecycle:** Create session → upload → ask → verify answer contains expected content.
2. **Invalid session:** Ask with a non-existent session ID → expect 404.
3. **No files uploaded:** Ask before uploading → expect 400.
4. **Unsupported file type:** Upload a `.exe` file → expect error response.
5. **Clear history:** Send messages → clear → verify history is empty.
6. **Multiple file upload:** Upload 3 files → verify all are indexed.
7. **Chunk count:** Upload a known document → verify expected chunk count.
8. **Integration test:** Full RAG pipeline with a mock LLM to verify end-to-end flow.
9. **Load test:** Use locust to test concurrent sessions.

Q48: How would you evaluate RAG quality?

RAG evaluation has two dimensions:

- **Retrieval quality:**
 - Precision@k: What fraction of retrieved chunks are relevant?
 - Recall@k: What fraction of relevant chunks were retrieved?
 - MRR (Mean Reciprocal Rank): Where does the first relevant chunk appear?
- **Generation quality:**
 - **Faithfulness:** Does the answer use only the retrieved context? (Use RAGAS framework.)
 - **Answer relevance:** Does the answer address the question?
 - **Groundedness:** Can every claim in the answer be traced to a source chunk?
- **Tools:** RAGAS, LangSmith evaluations, or custom evaluation pipelines with LLM-as-judge.

10.10 Scenario-Based Questions

Q49: A user uploads a 500-page PDF and the app becomes slow. How do you debug and fix this?

Diagnosis:

1. Profile the upload endpoint to find the bottleneck: is it file parsing, chunking, embedding, or ChromaDB insertion?
2. Most likely bottleneck: **embedding generation**—each chunk requires a forward pass through the transformer model.

Fixes:

1. **Batch embedding:** Embed chunks in batches (e.g., 64 at a time) instead of one by one.
2. **GPU acceleration:** Use CUDA-enabled PyTorch for the embedding model.
3. **Background processing:** Return immediately and process in a background task queue (Celery). Notify the user when indexing is complete.
4. **Progress feedback:** Send Server-Sent Events to the frontend showing “Indexing: 45/100 chunks...”.
5. **Caching:** Hash file content; if the same file was uploaded before, skip re-processing.

Q50: The LLM gives wrong answers even though the correct information is in the document. What could be wrong?

Possible root causes and fixes:

1. **Retrieval failure:** The relevant chunk wasn't in the top- k . *Fix:* Increase k , use hybrid search, or add re-ranking.
2. **Chunk boundary issue:** The answer spans two chunks and neither contains the full information. *Fix:* Increase chunk overlap, or use larger chunks.
3. **Embedding quality:** The embedding model doesn't capture the domain's vocabulary well. *Fix:* Use a domain-specific or larger embedding model.
4. **Prompt issue:** The prompt doesn't sufficiently constrain the model. *Fix:* Add instructions like "Quote the relevant text before answering."
5. **Context window overflow:** Too many chunks or too much history dilute the relevant context. *Fix:* Reduce history, use more focused retrieval.

Debug approach: Log the retrieved chunks for each query and inspect whether the correct information is being retrieved.

Q51: How would you make this work with multiple languages?

1. **Multilingual embeddings:** Replace `all-MiniLM-L6-v2` with a multilingual model like `paraphrase-multilingual-MiniLM-L12-v2` (supports 50+ languages).
2. **LLM:** Gemini already supports multiple languages natively.
3. **Document loaders:** The existing loaders are language-agnostic (they extract text regardless of language).
4. **Prompt:** Add "Answer in the same language as the question" to the system prompt.

Q52: A user uploads sensitive medical documents. What security measures would you implement?

1. **Encryption at rest:** Encrypt vector stores and session data using AES-256.
2. **Encryption in transit:** Enforce HTTPS with TLS 1.3.
3. **Access control:** Implement role-based access with JWT authentication.
4. **Data isolation:** Use separate vector store collections with per-user encryption keys.
5. **Audit logging:** Log all access to documents (who, when, what).
6. **Data retention:** Auto-delete sessions and data after a configurable TTL.
7. **HIPAA compliance:** Use a HIPAA-compliant cloud provider and sign a BAA.
8. **Local-only LLM:** Use a local model (e.g., Llama 3) instead of Gemini API to prevent data leaving the network.
9. **No temp files:** Process documents in-memory to avoid sensitive data on disk.

Q53: How would you add a feature to cite specific page numbers in the answer?

1. **Metadata preservation:** LangChain loaders (especially PyPDFLoader) automatically add `page` metadata to each `Document` object.
2. **Chunk metadata:** When splitting, metadata propagates to chunks. Each chunk knows its source file and page number.
3. **Retriever output:** The retrieved documents contain `metadata.page` and `metadata.source`.
4. **Prompt modification:** Include source info in the context: “[Page 5, file.pdf] chunk text...”.
5. **Response parsing:** Instruct the LLM to cite sources in its answer: “Based on page 5 of file.pdf...”.
6. **API response:** Return the source metadata alongside the answer for frontend rendering.

Q54: What happens if two users upload the same file in different sessions?

Each session has its own independent ChromaDB collection (`session_vectorstores[session_id]`). So the same file uploaded in two sessions creates **two separate sets of embeddings**. There's no deduplication. This is by design for session isolation but is wasteful. In production, you could:

1. Hash file content on upload.
2. Check if the hash exists in a shared embedding cache.
3. If it does, link the session to the existing embeddings instead of re-embedding.

Q55: How does the 20-message history limit prevent issues?

LLMs have a fixed context window (token limit). Each message in the chat history consumes tokens. Without a limit:

- The prompt would eventually exceed the LLM's context window, causing an error.
- Even before hitting the limit, long histories dilute the relevance of the current context, degrading answer quality.
- Token costs increase linearly with history length.

The 20-message limit (10 turns) is a sliding window—the oldest messages are dropped first. A more sophisticated approach would be to **summarize old history** into a condensed message rather than simply truncating it.

10.11 Conceptual / Theory Questions

Q56: What is the difference between RAG and fine-tuning?

Aspect	RAG	Fine-Tuning
Knowledge source	External database	Model weights
Update speed	Instant (add docs)	Requires retraining
Cost	Low (no training)	High (GPU hours)
Hallucination	Lower (grounded)	Higher
Traceability	Yes (cite sources)	No
Best for	Dynamic knowledge	Domain adaptation

Q57: What is an ASGI server and how does it differ from WSGI?

- **WSGI** (Web Server Gateway Interface): Synchronous protocol for Python web apps. One request blocks one thread. Used by Flask/Django with Gunicorn.
- **ASGI** (Asynchronous Server Gateway Interface): Async protocol supporting `async/await`, WebSockets, and long-lived connections. Used by FastAPI with Uvicorn.

ASGI is superior for I/O-bound workloads (like calling external LLM APIs) because a single process can handle many concurrent requests without blocking.

Q58: What is HNSW and how does it relate to vector search?

HNSW (Hierarchical Navigable Small World) is an approximate nearest neighbor (ANN) algorithm used by ChromaDB for efficient vector search. Instead of comparing the query vector against every stored vector ($O(n)$), HNSW builds a multi-layer graph where:

- The top layers have few, widely-spaced nodes for coarse search.
- Lower layers have more nodes for fine-grained search.
- Search starts at the top and “navigates” down, achieving $O(\log n)$ complexity.

For small datasets (<10K vectors), brute-force is fine. HNSW becomes critical for millions of vectors.

Q59: Explain the concept of “grounding” in LLMs.

Grounding means anchoring the LLM’s outputs to verified, factual sources rather than relying solely on its parametric (training) knowledge. In RAG:

- The retrieved document chunks serve as the “ground truth.”
- The system prompt instructs the model to use “ONLY the context below.”
- This reduces hallucination—the model generates answers from real data, not from patterns memorized during training.

Grounding is essential for domains where accuracy matters (medical, legal, academic).

Q60: What is the difference between semantic search and keyword search?

- **Keyword search** (BM25, TF-IDF): Matches exact terms. Searching “car” won’t find “automobile” or “vehicle.” Fast, interpretable, but misses meaning.
- **Semantic search** (vector embeddings): Matches meaning. “How to improve performance” matches “optimization techniques” even without shared keywords. Uses neural networks to encode meaning into vectors. Slower but far more powerful.

This project uses semantic search via all-MiniLM-L6-v2 embeddings + ChromaDB cosine similarity. Adding BM25 (hybrid search) would give the best of both worlds.

11 Future Improvements

1. **Streaming Responses:** Token-by-token streaming via SSE.
2. **Persistent Storage:** Redis for sessions, persistent ChromaDB or Pinecone for vectors.
3. **Authentication:** JWT-based auth with user accounts.
4. **Re-ranking:** Cross-encoder re-ranking for improved retrieval precision.
5. **Hybrid Search:** Combine dense and sparse retrieval.
6. **Source Citations:** Show page numbers and file names in answers.
7. **Multi-modal:** Support images and tables in documents (using Gemini’s vision).
8. **Evaluation Pipeline:** Automated RAG quality evaluation with RAGAS.
9. **LLM Observability:** LangSmith integration for tracing and debugging.
10. **Docker Deployment:** Dockerfile and docker-compose for one-command deployment.

12 Conclusion

The Document AI Chatbot demonstrates a production-quality RAG architecture using modern AI/ML tools. The project showcases proficiency in:

- **AI/ML:** RAG pipelines, embeddings, vector databases, LLM prompting, conversation memory.
- **Backend Engineering:** FastAPI, async Python, REST API design, error handling, session management.
- **Frontend Development:** Responsive UI, CSS design systems, DOM manipulation, file handling APIs.
- **Software Engineering:** Clean code, testing, dependency management, documentation.

The project balances practical engineering with AI innovation, making it an excellent portfolio piece and interview discussion topic.